

Tập 1 - Kubernetes Network Model

4 nguyên tắc không NAT & Sức mạnh của CNI

Phần 0 — Nền tảng K8s Networking · `#NetworkModel` `#CNI` `#routing`





Lộ trình bài học mới: Lab First - Slide Second

Chúng ta không bắt đầu bằng lý thuyết khô khan. Hôm nay, chúng ta sẽ đi theo mô hình lớp học đảo ngược:

- **PHẦN 1: Thực hành thực tế (Lab First)**
 - Khởi động cụm K8s "trắng" để xem sự cố xảy ra khi thiếu card mạng.
 - Tự tay cài đặt mạng (CNI) và chứng kiến sự thay đổi của Linux.
- **PHẦN 2: Đúc kết bản chất (Slide Second)**
 - Phân tích 4 nguyên tắc vàng của Kubernetes Network Model.
 - Khám phá cơ chế định tuyến (Routing) giải quyết bài toán không NAT.

PHẦN 1: Bắt đầu làm Lab ngay!

Chúng ta sẽ thực hiện 3 Thí nghiệm chuyên sâu trong file `lab-guide.md` :

1. **Thí nghiệm 1:** Chứng kiến cảnh cụm K8s tê liệt khi chưa cài CNI (Node `NotReady` , Pod bị kẹt ở `Pending` mãi mãi).
2. **Thí nghiệm 2:** Cài đặt Flannel CNI và xem các Node tự động "hồi sinh" chuyển sang `Ready` .
3. **Thí nghiệm 3:** Làn mò "dấu vết" mạng mà CNI thiết lập trên hệ điều hành (`flannel.1` card, `cni0` bridge và bảng routing).

👉 Hãy tạm dừng video, mở terminal và gõ lệnh theo hướng dẫn trong tệp `lab-guide.md` nhé!

PHẦN 2: Đúc kết từ Lab — Hiện trạng bạn vừa thấy

Sau khi làm Lab, chúng ta phát hiện ra 2 trạng thái trái ngược hoàn toàn:

1. Trạng thái "Vô danh" (Chưa cài CNI)

- Lệnh `kubectl get nodes` báo `NotReady`. Kubelet báo lỗi: `network plugin is not ready`.
- Card mạng trên Node chỉ có các cổng cơ bản (`eth0` , `lo`). Hoàn toàn không có card mạng nào cho Pod.

2. Sau khi cài CNI (Flannel)

- Mọi node chuyển sang `Ready` thần tốc. Pod chuyển `Running` và có IP riêng.
- Card mạng ảo mới xuất hiện: `cnio` (bridge) và `flannel.1` (VTEP VXLAN).
- Bảng định tuyến (`ip route`) xuất hiện dòng: `10.244.x.x` đi qua các card mạng ảo này.

Hợp đồng mạng: Kubernetes Network Model

Kubernetes **không tự cài mạng**, nó chỉ đặt ra một **hợp đồng bắt buộc** gồm 4 nguyên tắc vàng mà bất kỳ CNI nào cũng phải tuân thủ:

Nguyên tắc 1: Pod-to-Pod không NAT (dù khác Node)

Pod A IP: 10.244.1.5 → Pod B IP: 10.244.2.7 (Giữ nguyên IP nguồn)

Nguyên tắc 2: Node-to-Pod không NAT

Worker node IP: 192.168.64.11 → Pod IP: 10.244.2.7 (Đến thẳng trực tiếp)

Nguyên tắc 3: Pod thấy đúng IP nguồn của caller

Pod B nhận gói tin: src_ip = 10.244.1.5 (Không bị NAT thành IP của Node)

Nguyên tắc 4: Pod IP là độc bản trên toàn bộ Cluster

Không bao giờ có 2 Pod trùng IP nhau, dù chúng nằm ở bất kỳ đâu

Tại sao quy tắc "Không NAT" lại khó?

- **Mạng thông thường (Có NAT):**

Router chỉ cần định tuyến cho IP của máy Host (`192.168.x.x`). Traffic đi ra ngoài sẽ bị đổi IP nguồn (Masquerade) thành IP Host. Cực kỳ đơn giản cho thiết bị mạng vật lý.

- **Mạng Kubernetes (Không NAT):**

Gói tin đi từ Pod A sang Pod B bắt buộc phải giữ nguyên địa chỉ `10.244.1.5` làm IP nguồn.

- 🖱️ **Giải pháp:** Router vật lý phải biết đường đi của các dải mạng con `10.244.x.x` . CNI giải quyết việc này bằng 2 cách:

- i. **Direct Routing** (Ví dụ Calico, Flannel host-gw): Quảng bá bảng định tuyến.

- ii. **Encapsulation/Overlay** (Ví dụ Flannel VXLAN): Bọc gói tin của Pod vào trong gói tin của Node (giống như gửi một bức thư bên trong một hộp bưu phẩm khác).

Key Takeaways — Bài học cốt lõi

- **CNI không phải là phép thuật:** Nó thực chất chỉ là một tiến trình tự động thực hiện các câu lệnh Linux cơ bản: Tạo card mạng ảo (`cni0`, `flannel.1`) và cấu hình bảng định tuyến (`ip route`).
- **Mạng K8s là mạng phẳng (Flat Network):** Mọi Pod có thể giao tiếp trực tiếp với nhau qua IP của nó mà không cần thông qua NAT.
- **IP của Pod là duy nhất:** CNI chịu trách nhiệm phân chia các dải mạng con (Subnet) cho mỗi Node để không bao giờ xảy ra conflict IP.

Tập tiếp theo: Ai đã cắm dây cáp ảo nối Pod ra ngoài thế giới? Pause container đóng vai trò gì? Hãy cùng khám phá `veth pair` !